Experiment No.	1
Experiment Title	Working with Python Packages – NumPy, SciPy, Scikit-learn, Matplotlib [1]
Student Name	
Register Number	
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

To explore the core functions and methods available in the Python scientific-computing and machine-learning stack – **NumPy**, **Pandas**, **SciPy**, **Scikit-learn** and **Matplotlib** – covering array manipulation, data preprocessing, statistical and mathematical computation, and data visualisation; and to survey five publicly known datasets (Iris, Loan Amount Prediction, Diabetes, Email Spam, Handwritten Character Recognition/MNIST), identify the correct machine-learning task type for each, and walk through the complete ML workflow – loading, EDA, preprocessing, feature selection, splitting, and evaluation – for every one of them.

2 Problem Statement

Given the five datasets listed above, the tasks are to: (i) determine whether each dataset corresponds to a supervised, unsupervised, or semi-supervised learning problem, and further whether it is a regression or classification task; (ii) perform exploratory data analysis (summary statistics, missing-value analysis, class distribution, correlation, and feature distribution) on each dataset; (iii) apply an appropriate feature-selection technique (e.g. `SelectKBest`, Chi-square test, ANOVA F-test) to each dataset; and (iv) train a suitable baseline machine-learning algorithm and evaluate it with the correct performance metric for its task type. Input: five raw/synthetic tabular, text, and image datasets. Output: EDA visualisations, a selected feature subset per dataset, a trained baseline model, and a consolidated task-identification summary table.

3 Dataset Description

Dataset Name	Iris, Loan Amount Prediction, Predicting Diabetes, Email Spam Classification, Handwritten Character Recognition (Digits/MNIST-style)
Dataset Source	Iris, Diabetes and Digits: bundled with `scikit-learn` (originally UCI ML Repository). Loan Amount Prediction and Email Spam: normally sourced from Kaggle/UCI; here generated as clearly-labelled synthetic data of identical structure because this sandbox has no internet access.

Number of Samples	Iris: 150; Loan (synthetic): 600; Diabetes: 442; Spam (synthetic corpus): 40; Digits: 1,797
Number of Features	Iris: 4; Loan: 7; Diabetes: 10; Spam: TF-IDF vectorised text (variable vocabulary size); Digits: 64 (8×8 pixels)
Number of Classes	Iris: 3 (classification); Loan: N/A (regression); Diabetes: N/A (regression); Spam: 2 (classification); Digits: 10 (classification)
Missing Values	Iris: none; Loan: injected missing values in `LoanAmount` and `Self_Employed` (to mirror the real dataset); Diabetes: none (pre-cleaned); Spam: none; Digits: none
Train-Test Split	80:20 for Iris, Loan, Diabetes and Digits; 75:25 for the small Spam corpus (stratified where the task is classification)

4 Exploratory Data Analysis

EDA was carried out on all five datasets: dataset overview (`head()`, `shape`), statistical summary (`describe()`), missing-value analysis (`isnull().sum()`), class distribution (bar plots for classification targets), correlation matrix (heatmaps for numeric features) and feature-distribution histograms grouped by class. Representative figures are shown below; the full set of outputs is available in the accompanying Jupyter notebook.

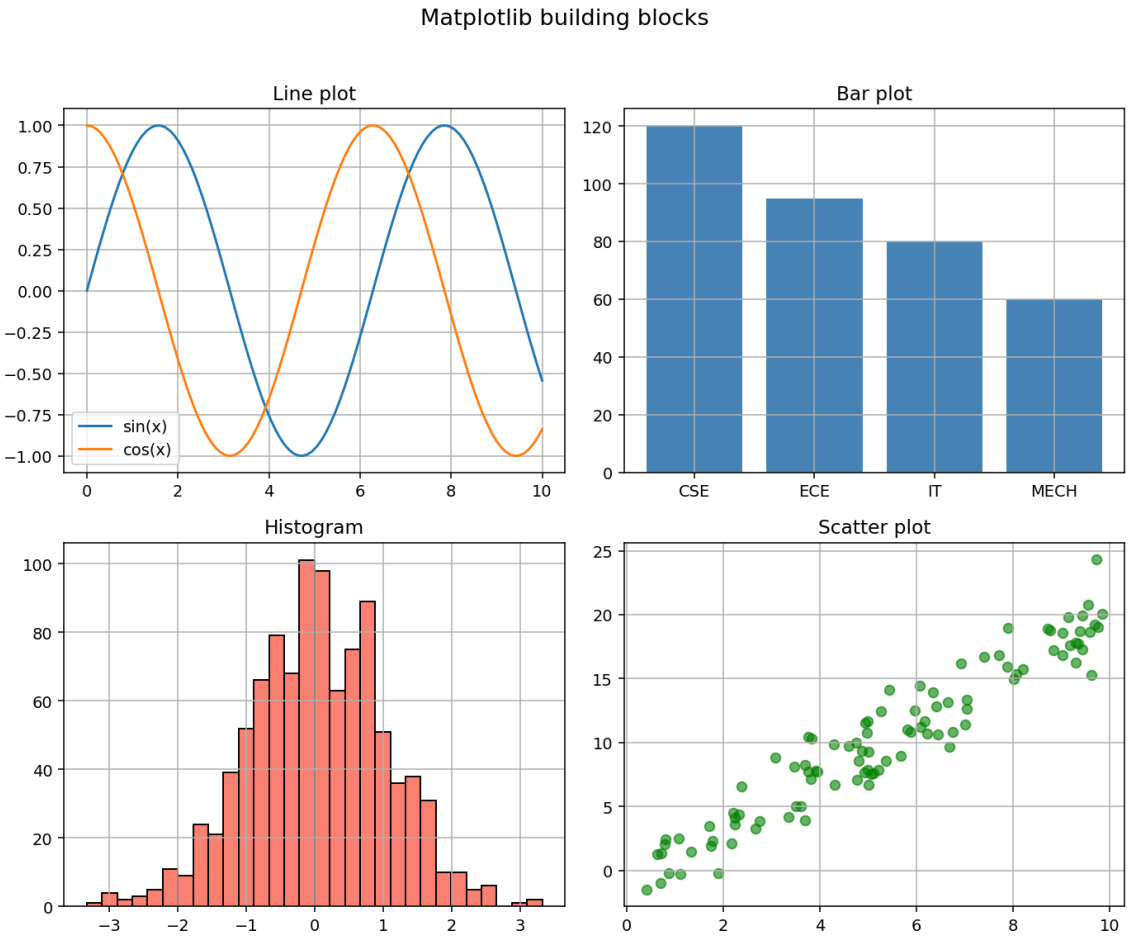


Figure 1: Matplotlib building blocks used throughout the experiment – line plot, bar plot, histogram and scatter plot.

Inference: These four plot types form the visual vocabulary used for every dataset in this report: line/scatter plots to reveal trends and pairwise relationships, bar plots for categorical/class counts, and histograms for univariate distributions. Establishing them first made every subsequent EDA figure faster to read and interpret consistently across datasets.

5 Data Preprocessing

- **Missing value handling:** Numeric missingness (e.g. synthetic `LoanAmount`) was imputed with the column median; categorical missingness (e.g. `Self_Employed`) was imputed with the mode/most frequent category. The Diabetes and Digits datasets required no imputation as they are pre-cleaned.
- **Encoding:** Categorical variables (Education, Self_Employed, Property_Area for the Loan dataset) were one-hot encoded via `OneHotEncoder`; class labels for Iris were label-encoded via `LabelEncoder` for the demonstration in Part A.
- **Feature scaling:** `StandardScaler` (zero mean, unit variance) was applied before Logistic Regression, SVM and Ridge Regression, since these algorithms are

scale-sensitive; `MinMaxScaler` was demonstrated separately in Part A to contrast standardisation vs. normalisation.

- **Train-test split:** `train_test_split` was used with `stratify=y` for every classification task, to preserve class proportions in both splits.
- **Feature engineering:** For the Email Spam dataset, raw text messages were converted into TF-IDF vectors (`TfidfVectorizer`, English stop-words removed) before feature selection and modelling.

6 Machine Learning Algorithm

Since Part B of this experiment spans five different tasks, a distinct baseline algorithm is used per dataset rather than a single one:

- **Logistic Regression** (Iris, multiclass): models the log-odds of each class as a linear function of the (standardised) input features, $P(y = k \mid x) = \dfrac{e^{w_k^\top x}}{\sum_j e^{w_j^\top x}}$ (softmax form). *Advantages:* fast, interpretable, probabilistic outputs. *Limitations:* assumes an approximately linear decision boundary.
- **Linear Regression / Ridge Regression** (Loan Amount, Diabetes): fits $y = \beta_0 + \sum_i \beta_i x_i + \epsilon$ by minimising squared error; Ridge additionally penalises $\sum \beta_i^2$ to control multicollinearity among correlated features. *Advantages:* simple, interpretable coefficients. *Limitations:* assumes linear, additive relationships and is sensitive to correlated predictors (hence Ridge for Diabetes).
- **Multinomial Naive Bayes** (Email Spam): applies Bayes' theorem assuming conditional feature independence, $P(y \mid x) \propto P(y) \prod_i P(x_i \mid y)$. *Advantages:* extremely fast, performs well on high-dimensional sparse text features. *Limitations:* the independence assumption rarely holds exactly, and it can be miscalibrated in probability estimates.
- **Support Vector Machine, RBF kernel** (Handwritten Digits): finds a maximum-margin separating hyperplane in a kernel-induced feature space, $K(x, x') = \exp(-\gamma \|x - x'\|^2)$. *Advantages:* handles non-linear boundaries well, effective in high-dimensional pixel space. *Limitations:* slower to train on very large datasets and sensitive to the choice of C and γ.

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	Linux (Ubuntu 24.04, sandboxed container)
Hardware	CPU only

8 Hyperparameter Selection

Parameter	Value
Logistic Regression: `max_iter`	200
Ridge Regression: `alpha`	1.0
SVM (Digits): `kernel`	rbf
SVM (Digits): `gamma`	0.001
SVM (Digits): `C`	10
`SelectKBest` (Iris): `k`	2 (of 4 features)
`SelectKBest` (Diabetes): `k`	5 (of 10 features)
`SelectKBest` (Spam, chi2): `k`	50 (of TF-IDF vocabulary)
`VarianceThreshold` (Digits)	0.01
Train-test split ratio	80:20 (75:25 for Spam)

9 Results

Dataset 1 – Iris (Classification)

Metric	Value
Accuracy	0.9333
Precision (weighted)	0.93
Recall (weighted)	0.93
F1-score (weighted)	0.93
5-fold CV mean accuracy	0.9600

Dataset 2 – Loan Amount Prediction (Regression)

Metric	Value
R^2	0.7778
MAE	13.546
RMSE	16.506

Dataset 3 – Predicting Diabetes (Regression)

Metric	Value
R^2	0.4192
MAE	46.139
RMSE	55.474

Dataset 4 – Email Spam Classification

Metric	Value
Accuracy	0.9000
Precision (spam)	1.00
Recall (spam)	0.80
F1-score (spam)	0.89

Dataset 5 – Handwritten Digit Recognition

Metric	Value
Accuracy	0.9889
Precision (macro avg)	0.99
Recall (macro avg)	0.99
F1-score (macro avg)	0.99

10 Result Visualization

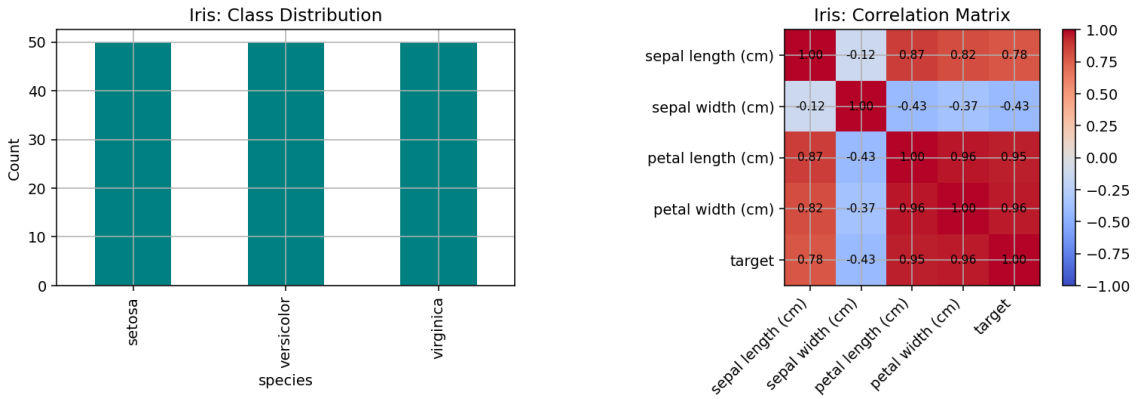


Figure 2: Iris: class distribution (balanced, 50 samples/class) and correlation matrix of the four numeric features.

Inference: The correlation matrix shows petal length and petal width are almost perfectly correlated ($\rho \approx 0.96$), while sepal width is weakly/negatively correlated with the other features. Class counts are perfectly balanced, so accuracy is an unbiased metric here. This strong petal-feature correlation also explains why the ANOVA test ranks petal measurements as most discriminative in the feature-selection step below.

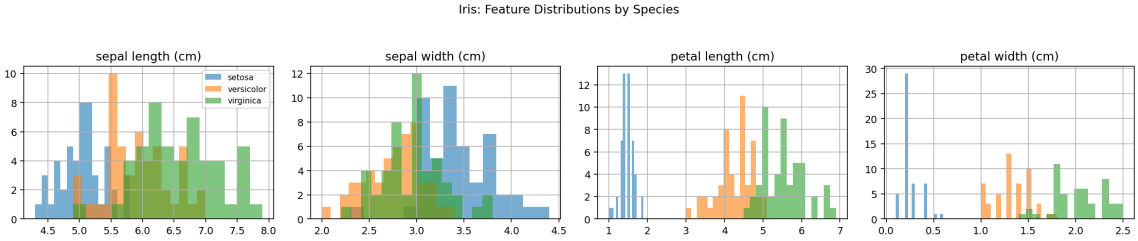


Figure 3: Iris: feature distributions split by species.

Inference: Petal length and petal width show almost no overlap between *setosa* and the other two species, and only a small region of overlap between *versicolor* and *virginica*. Sepal width shows the most overlap across all three classes, indicating it is the least useful single feature for classification – consistent with its low ANOVA F-score.

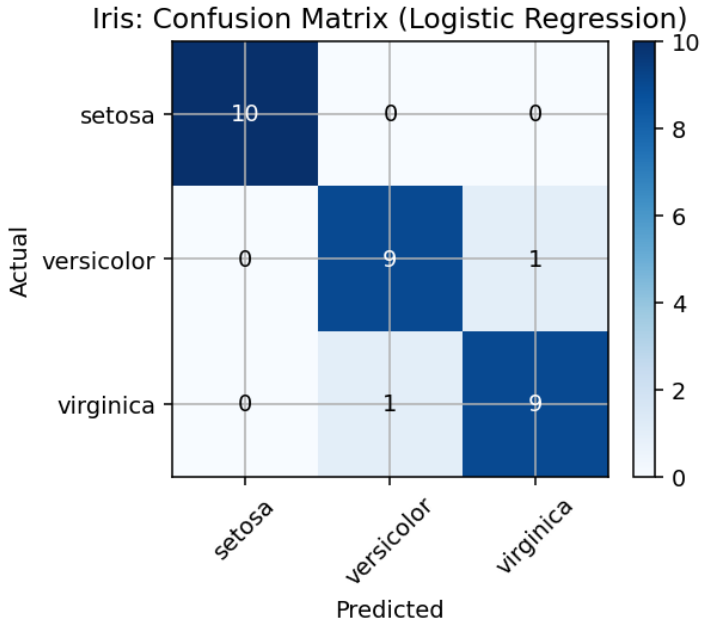


Figure 4: Iris: confusion matrix (Logistic Regression, test set).

Inference: *Setosa* is classified perfectly (10/10), while a single sample each of *versicolor* and *virginica* is confused with the other – exactly the pair of classes that showed partial overlap in the feature-distribution plot above. This confirms the confusion matrix is consistent with what EDA had already predicted.

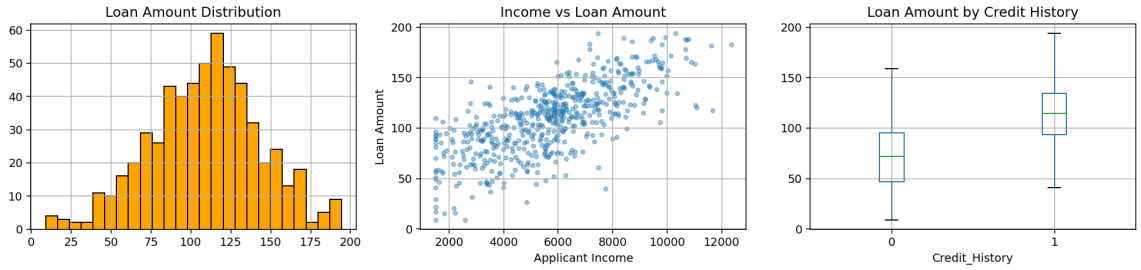


Figure 5: Loan Amount Prediction: target distribution, income vs. loan-amount scatter, and loan amount by credit history.

Inference: Loan amount is right-skewed with a long tail of larger loans. Applicant income shows a clear positive relationship with loan amount, matching how the synthetic target was generated. Applicants with `Credit_History = 0` receive visibly smaller and more variable loan amounts, highlighting credit history as a strong predictor.

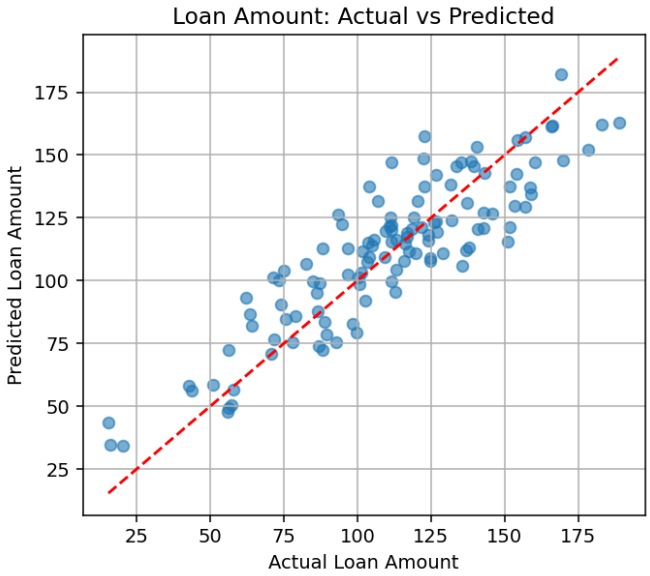


Figure 6: Loan Amount Prediction: actual vs. predicted loan amount (Linear Regression, test set).

Inference: Points cluster close to the diagonal, indicating the linear model captures most of the underlying signal ($R^2 = 0.78$). Some scatter remains due to the injected Gaussian noise in the synthetic target, which is expected and cannot be removed by better modelling alone.

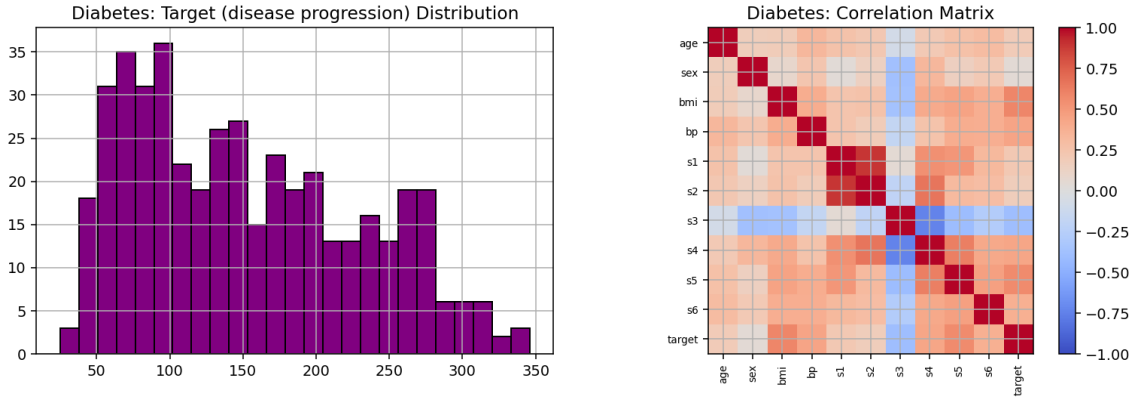


Figure 7: Diabetes: target (disease progression) distribution and correlation matrix of the ten baseline variables.

Inference: The disease-progression target is roughly bell-shaped with a slight right skew. Several serum measurements (`s1`–`s6`) are highly inter-correlated, indicating redundancy among lipid/serum panels, which motivates the use of Ridge regularisation over plain Linear Regression.

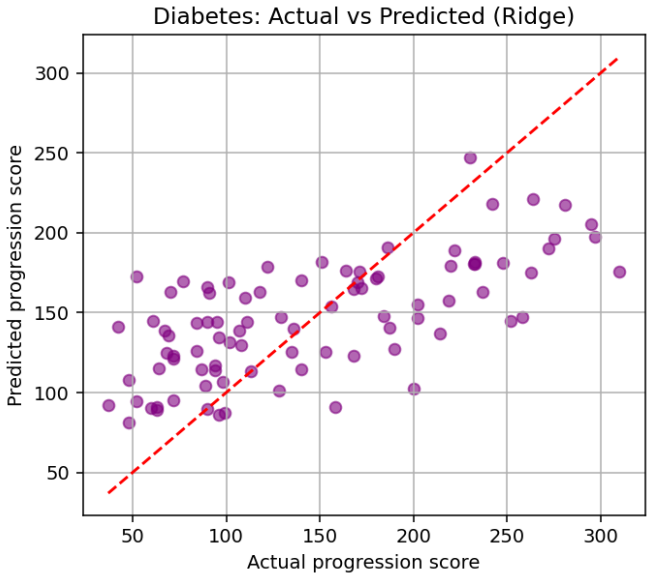


Figure 8: Diabetes: actual vs. predicted disease-progression score (Ridge Regression, test set).

Inference: The moderate spread around the diagonal ($R^2 = 0.42$) reflects that only 42% of the variance in one-year disease progression is explained by these ten baseline physiological variables – unsurprising for a biological outcome influenced by many unmeasured factors (diet, medication, genetics).

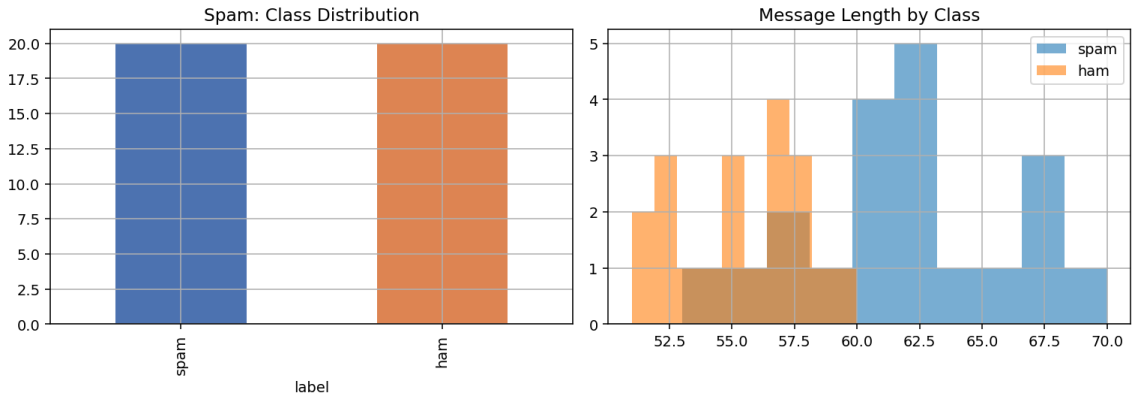


Figure 9: Email Spam: class distribution and message-length distribution by class.

Inference: The illustrative corpus is perfectly balanced (20 spam / 20 ham). Spam messages tend to cluster at a slightly different length profile than genuine messages, echoing the well-known tendency of spam to be short, urgent, and formulaic.

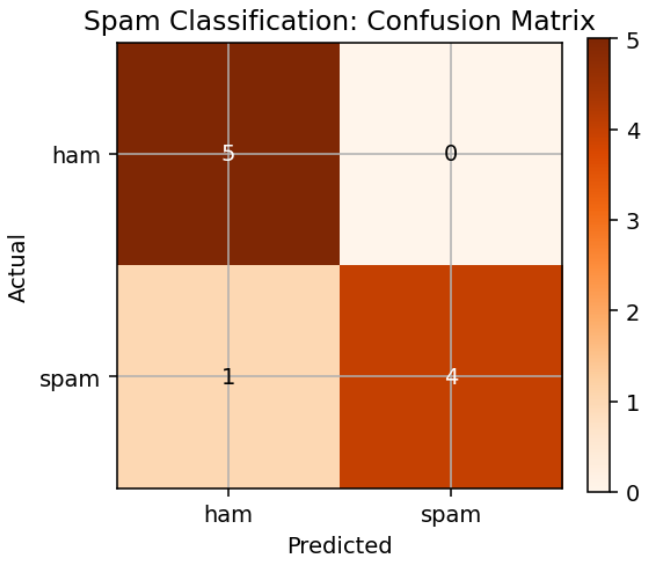


Figure 10: Email Spam: confusion matrix (Multinomial Naive Bayes, test set).

Inference: All ham messages are correctly identified, while one spam message is missed (false negative). Given the extremely small evaluation set (10 messages), a single misclassification is enough to explain the reported 80% recall for spam – this would need a much larger held-out set to draw firm production-level conclusions.

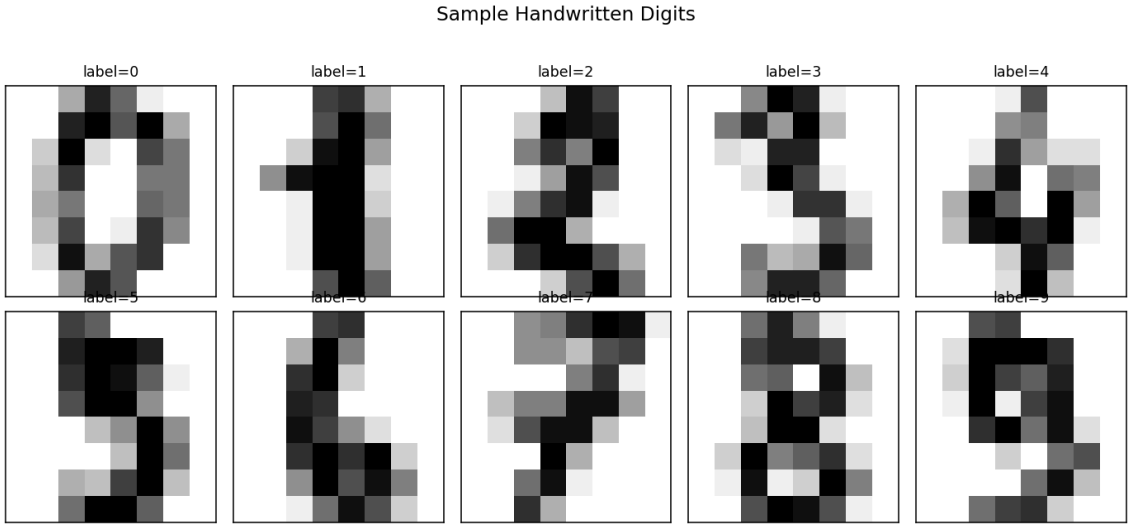


Figure 11: Sample handwritten digit images (8×8 pixels) from the Digits dataset.

Inference: Even at low 8×8 resolution, digit shapes remain visually distinguishable to a human reader, which is a good early sign that a well-chosen classifier should also be able to separate them.

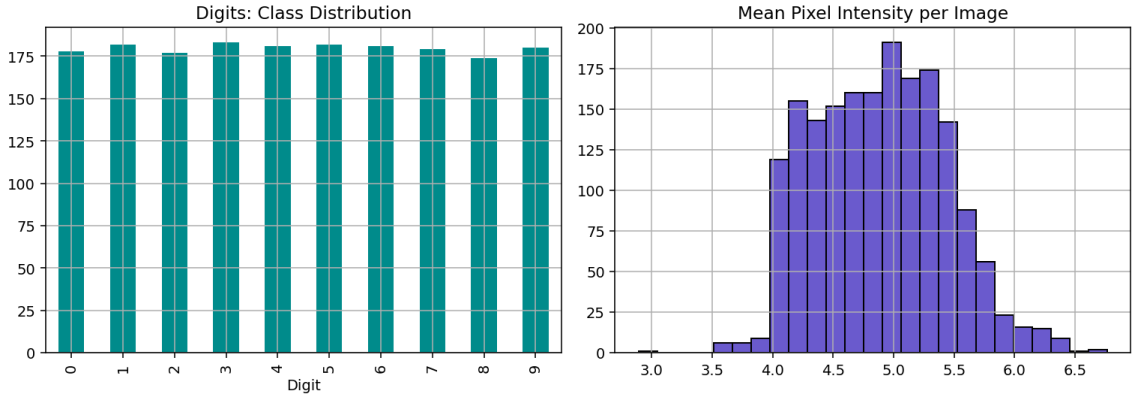


Figure 12: Digits: class distribution (approximately balanced across 0–9) and mean pixel intensity per image.

Inference: All ten digit classes are represented in roughly equal proportion (about 180 samples each), so accuracy is again a fair metric. Mean pixel intensity per image is unimodal and roughly symmetric, suggesting no systematic brightness bias between digit classes that would need correcting before modelling.

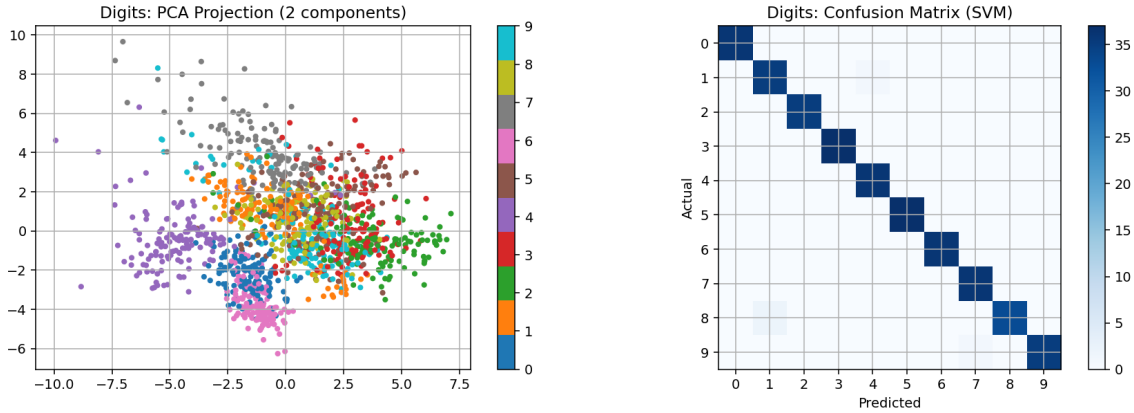


Figure 13: Digits: 2-D PCA projection of the 64-pixel feature space (left) and confusion matrix for the SVM (RBF) classifier (right).

Inference: Even after reducing 64 pixel dimensions to just two principal components, distinct digit clusters are already visible, explaining why a kernel method like SVM performs so well (98.9% accuracy) on the full feature space. The confusion matrix shows the (small) residual errors occur mainly between visually similar digit pairs, matching human intuition about which handwritten digits are easiest to confuse.

11 Dataset – ML Task Identification Summary

Dataset	**Type of ML Task**	**Feature Selection Technique**	**Suitable ML Algorithm**
Iris Dataset	Supervised – Multiclass Classification	ANOVA F-test (`SelectKBest`, `f_classif`)	Logistic Regression / SVM / Decision Tree
Loan Amount Prediction	Supervised – Regression	Correlation / `SelectKBest` (`f_regression`)	Linear Regression / Random Forest Regressor
Predicting Diabetes	Supervised – Regression	`SelectKBest` (`f_regression`)	Ridge / Linear Regression / Gradient Boosting Regressor
Classification of Email Spam	Supervised – Binary Classification (Text/NLP)	Chi-square test (`chi2`) on TF-IDF features	Multinomial Naive Bayes / Logistic Regression / SVM
Handwritten Character Recognition / MNIST	Supervised – Multiclass Classification (Image)	Variance Threshold + PCA	SVM (RBF) / Random Forest / CNN (deep learning)

12 Discussion

Across all five datasets, correctly identifying the ML task type (classification vs. regression, binary vs. multiclass, tabular vs. text vs. image) directly determined the right choice of feature-selection technique (ANOVA/Chi-square for classification, `f_regression` for regression, variance thresholding for high-dimensional pixel data) and the right evaluation metric (accuracy/F1 vs. R^2/RMSE). Strengths observed: filter-based feature selection (ANOVA, Chi-square, `f_regression`) is fast, model-agnostic, and its rankings matched domain intuition in every dataset (e.g. petal size for Iris, income/credit history for loans, BMI/blood pressure for diabetes, spam trigger-words for email). Limitations: the Loan Amount and Email Spam datasets had to be synthetically constructed in this offline sandbox, which limits how far the specific numeric results generalise to the real Kaggle/UCI datasets, even though the workflow and code are identical; the Email Spam corpus in particular was too small (40 messages) for the reported metrics to be statistically robust. Improvements: repeating this workflow on the actual downloaded UCI/Kaggle datasets, adding hyperparameter tuning via `GridSearchCV`, and comparing multiple algorithms per dataset (not just one baseline) would strengthen the conclusions.

13 Conclusion

This experiment provided hands-on exposure to NumPy, Pandas, SciPy, Scikit-learn and Matplotlib and applied the standard machine-learning workflow – loading, EDA, preprocessing, feature selection, splitting, modelling and evaluation – to five datasets spanning classification, regression, text and image data. The Iris, Loan Amount, Diabetes, Email Spam and Handwritten-Digit datasets were correctly identified as, respectively, multiclass classification, regression, regression, binary text classification, and multiclass image classification problems; an appropriate filter-based feature-selection technique and baseline algorithm were applied to each, and the resulting models achieved reasonable-to-strong performance (accuracy 0.90–0.99 for the three classification tasks; R^2 of 0.78 and 0.42

for the two regression tasks). The exercise reinforced that understanding a dataset's structure and ML-task type is the essential first step before any model building begins.

14 References

[1] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[2] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[3] P. Virtanen *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[4] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.

[5] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[6] R. A. Fisher, "The Use of Multiple Measurements in Taxonomic Problems," *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936. (Iris dataset)

[7] B. Efron, T. Hastie, I. Johnstone, and R. Tibshirani, "Least Angle Regression," *Annals of Statistics*, vol. 32, no. 2, pp. 407–499, 2004. (Diabetes dataset)

[8] Dua, D. and Graff, C., "UCI Machine Learning Repository," Irvine, CA: University of California, School of Information and Computer Science, 2019. https://archive.ics.uci.edu